

SIH PS 26037 — Deep Research & Technical Analysis

Adaptive Path Planning and Collision Avoidance for Autonomous Vehicles on Unstructured Indian Roads

Organization: MathWorks | **Category:** Software | **Theme:** Smart Vehicles

Executive Summary

PS 26037 asks teams to build a *simulated, closed-loop* adaptive path-planning and collision-avoidance system for an autonomous vehicle operating on unstructured Indian roads — roads without reliable lane markings, with heterogeneous traffic (cars, buses, auto-rickshaws, two-wheelers, bicycles, pedestrians, pushcarts, and animals), informal merging, and unsignalized intersections. The deliverable is a MATLAB/Simulink simulation pipeline (perception → prediction → planning → decision → control), validated across five named scenarios, with quantitative metrics, a report, and a demo video.

The research below shows this is a well-posed but non-trivial systems-integration problem, not an open research problem requiring new algorithms. The genuine technical difficulty is **not** "autonomous driving is hard" in the abstract — it is that mainstream planning stacks (Apollo, Autoware, most published motion planners) are built around structured assumptions: lane graphs/HD maps, lane-based behavior prediction, and traffic-rule-conditioned decision logic. Autoware and Apollo both organize their planning modules around lane- and scenario-based execution, selecting from predefined driving scenarios such as lane-following or parking). Published trajectory-prediction benchmarks that specifically target Indian-style heterogeneous, non-lane-based traffic (TrafficPredict, TraPHic, IDD/IDD-3D) exist and quantify this failure mode directly, giving us a defensible, evidence-backed gap rather than a manufactured one.

Our recommended solution is **not** a novel algorithm. It is a defensible **integration and adaptation**: a sampling/search-based planner (Hybrid A* for structured-ish segments, informed RRT* for the unstructured/off-lane cases) fused with a lightweight occupancy-grid + costmap environment representation (rather than a lane-graph), a hybrid classical+learned short-horizon prediction module that is category-aware (vehicle / two-wheeler / pedestrian / animal), and a Stateflow-based behavior arbitration layer, closed with a kinematic bicycle model and MPC/pure-pursuit tracking, all inside MATLAB/Simulink with RoadRunner-authored scenes. This is realistic for a 36-hour SIH build, is defensible to judges section-by-section, and is honestly scoped between "must have" and "future work."

1. Problem Statement

(As given — see Part A of the source brief; reproduced in full in the appendix/source document, not restated here to avoid duplication.) In one line: **design and simulate, in MATLAB/Simulink, a perception-to-control pipeline that lets an autonomous vehicle navigate unmarked, mixed-traffic Indian roads safely and replan in real time, validated on five representative Indian scenarios.**

2. Problem Understanding — 5-Minute Method

One-sentence problem formulation:

Autonomous vehicle path-planning stacks that assume structured road geometry (lane graphs, marked lanes, signalized intersections, lane-disciplined agents) fail when deployed on unstructured Indian roads — where lane markings are absent, road edges are unclear, and heterogeneous agents (auto-rickshaws, two-wheelers, pedestrians, pushcarts, animals) move in non-lane-based, unpredictable ways — resulting in unsafe, indecisive, or infeasible planning behavior and an inability to validate autonomy readiness for Indian deployment.

This satisfies the "specific stakeholder / specific problem / specific failure condition / specific consequence" test requested in the brief, and every subsequent section traces back to it.

3. Stakeholder Analysis

Stakeholder	Role	Why they experience the problem
Autonomous vehicle OEMs / AV software teams (primary)	Build planning stacks meant for eventual Indian deployment	Existing lane-based planners and prediction models were validated on structured-road datasets (nuScenes, Waymo Open, Argoverse) and degrade on Indian-style unstructured, heterogeneous traffic
MathWorks (problem owner)	Provider of the simulation toolchain (RoadRunner, Automated Driving Toolbox, Navigation Toolbox)	Wants evidence that its toolchain can model and validate Indian-specific edge cases, not just Western structured-road ADAS scenarios
End road users (pedestrians, cyclists, cart-pullers, animal owners)	Share the road space with the AV	Bear the safety consequences if prediction/planning misjudges their motion

Regulators / transport authorities (downstream)	Certify AV readiness for Indian roads	Need scenario-based, metric-backed evidence rather than marketing claims before permitting deployment
SIH judges (immediate stakeholder for this exercise)	Evaluate technical depth and feasibility	Need a defensible chain from problem → evidence → design decision, not a generic "AI self-driving car" demo

The PS's "vehicle" is not consumer-owned; it is closer to a research/pilot AV platform (the kind Indian AV pilot programs, university labs like IIIT-Hyderabad/iHub-Data, or logistics/last-mile players would operate), since full L4 personal AV deployment on Indian roads is not close to happening in 2026.

4. Current Situation

What exists today

- Open-source full stacks:** Autoware (ROS-based) and Apollo (Baidu) are the two dominant open-source autonomous-driving software stacks, each providing perception → localization → planning → control pipelines Autoware is an open-source software stack for self-driving vehicles built on ROS, including localization, object detection, route planning, and control). A direct benchmarking study found Autoware's Open Planner favors larger safety clearances at the cost of longer paths and higher energy use, while Apollo's Public Road planner produces smoother, more comfortable trajectories at higher computational cost Autoware.AI's Open Planner provided higher safety margins by maintaining larger clearance distances from obstacles, resulting in increased path lengths and energy consumption, while Apollo.Auto's Public Road planner offered smoother trajectories and enhanced passenger comfort but required higher computational resources). Both are architected around scenario selection (lane-following, parking, intersection) Autoware computes multiple scenarios such as lane and parking in parallel and activates the appropriate one based on vehicle context, while Apollo sequentially integrates traffic regulations before selecting a single scenario) — i.e., both assume a scenario library keyed to structured road types.
- MathWorks toolchain (encouraged by the PS):** RoadRunner is a purpose-built 3D road-network editor supporting custom road damage, potholes, and surface features via OpenCRG RoadRunner lets you insert guardrails, road damage, foliage, buildings, and other 3D models, and supports creating synthetic road geometries such as speedbumps and potholes), and RoadRunner Scenario lets actor behavior be authored in MATLAB/Simulink RoadRunner Scenario lets you customize the behavior of actors, including vehicles and pedestrians, using MATLAB and Simulink). Automated Driving Toolbox provides costmap-based path planning and lateral/longitudinal tracking controllers Automated Driving Toolbox lets you plan driving paths with vehicle costmaps and motion-planning algorithms, and use lateral and longitudinal controllers to follow a planned trajectory). Navigation Toolbox supplies the generic sampling/search planners and path-following/replanning tools Navigation Toolbox includes customizable search- and sampling-based path planners along with metrics for validating and comparing paths, and supports global and local path planning, path following, and obstacle avoidance).
- Indian-context datasets exist but are narrow:** IDD (10,004 images, 34 classes, 182 drive sequences) targets semantic segmentation on unstructured Indian roads IDD consists of 10,004 images finely annotated with 34 classes collected from 182 drive sequences on Indian roads, with an expanded label set compared to Cityscapes to account for new classes including drivable areas). IDD-3D extends this to multi-modal camera+LiDAR 3D detection/tracking with 12–15.5k annotated frames IDD-3D provides multi-modal data from multiple cameras and LiDAR sensors with roughly 12–15.5k annotated driving frames across various traffic scenarios, aimed at 3D object detection and tracking benchmarks in complex layouts), and explicitly annotates classes such as animals and mixer machines on roads that Western datasets do not label at all. Newer datasets like DriveIndia (66,986 images, 24 categories, 120+ hours, 3,400+ km) push this further for 2D detection DriveIndia is a large-scale object-detection dataset built to capture Indian traffic complexity, containing 66,986 high-resolution images annotated across 24 traffic-relevant categories, collected over 120+ hours and 3,400+ kilometers across urban, rural, and highway routes). None of these datasets provide planning or control ground truth — they are perception-only.
- Trajectory prediction for heterogeneous/Indian-style traffic is an active but narrow research niche.** TrafficPredict was among the first LSTM-based models to jointly predict pedestrians, bicycles, and vehicles, showing that treating agent categories separately with a shared interaction model improves over naive baselines TrafficPredict is the first method to integrate trajectory prediction for different kinds of traffic agents such as vehicles, bicycles, and pedestrians using a self-attention category layer). TraPHic goes further, explicitly modeling dense, heterogeneous Indian-style traffic including cars, buses, pedestrians, two-wheelers, three-wheelers, and animals, using a weighted LSTM-CNN interaction model, and reports up to 30% improvement over prior trajectory-prediction methods on this traffic mix TraPHic is an end-to-end approach for dense and heterogeneous traffic that models interactions between cars, buses, pedestrians, two-wheelers, three-wheelers, and animals without assuming lane-based driving, achieving up to 30% improvement in prediction accuracy over prior methods).

Why it is insufficient for this PS

The gap is not "no research exists" — it is that (a) no single open pipeline joins Indian-context perception, Indian-context prediction, and unstructured-road planning into one closed-loop, controllable simulation; and (b) the full AV stacks (Apollo/Autoware) that do close the loop are structurally lane-centric, and adapting them for a 36-hour SIH build is a much larger integration burden than building a MATLAB/Simulink pipeline directly on top of the toolboxes the PS already names.

5. Pain Point

Teams (and, at the level above SIH, real AV developers) cannot currently point to a **closed-loop, quantitatively validated demonstration** that a planning system remains safe and functional when lane markings disappear and traffic composition becomes heterogeneous and non-lane-based. Today's public evidence is split: perception datasets show *what* Indian roads look like; prediction papers show *that* heterogeneous-traffic prediction is harder and quantify

the gap; but there is no widely cited open closed-loop planning+control benchmark that ties detection → prediction → planning → control together specifically for Indian-style unstructured roads with the five scenario types the PS names.

6. Failure Condition (verified, not assumed)

Condition named in PS	Verified as a genuine technical failure mode?	Evidence
Missing/no lane markings	Yes	IDD and IDD-3D were built specifically because existing (lane-marking-dependent) segmentation/detection datasets did not generalize to Indian roads IDD reflects label distributions of road scenes significantly different from existing datasets, with most classes displaying greater within-class diversity, and introduces new classes such as drivable areas).
Heterogeneous, non-lane-based mixed traffic (auto-rickshaws, two-wheelers, pushcarts, animals)	Yes	TraPHic and TrafficPredict were built precisely because homogeneous-agent prediction models fail on this mix A benchmark for heterogeneous traffic found that as traffic heterogeneity increases, state-of-the-art forecasting and planning methods show significant performance degradation, with two-wheelers being the hardest agent type to predict accurately).
Unsignalized intersections / informal merging	Yes, but less India-specific	This is a generally studied AV problem (unsignalized-intersection negotiation), not unique to India; treat as "strongly implied, moderately researched" rather than a novel gap.
Sudden pedestrian / animal crossing	Yes	Roadway-animal detection is a distinct, actively researched sub-problem the paper devises a real-time detection method for Indian stray and wild animals on roadways to help prevent collisions) and is not covered at all by standard Western AV perception classes.
Lane-based planners (Apollo/Autoware-style) failing outright vs. degrading	Degrading, not necessarily failing outright — important nuance	The Autoware/Apollo comparison shows both function but trade off safety margin vs. comfort/efficiency even in relatively structured tests Autoware.AI's Open Planner provided higher safety margins... Apollo.Auto's Public Road planner offered smoother trajectories... but required higher computational resources); extrapolating "these fail completely on Indian roads" is an assumption, not a demonstrated fact, since neither was tested against Indian-style scenes in the source study.

Conclusion: the failure condition is real and evidenced for (1) unstructured road geometry perception, and (2) heterogeneous/non-lane-based agent prediction. It is *plausible but not directly evidenced* for full closed-loop planning failure — we should say "expected to degrade based on lane-centric architecture" rather than "proven to fail," to stay honest with judges.

7. Desired Outcome

Measurable success = the PS's own listed criteria, refined:

- **Collision-free completion rate** across all 5 scenarios, across multiple randomized runs each (not a single run).
- **Replanning latency** (time from new obstacle detection to updated trajectory available) — must be within a real-time budget (target: sub-200 ms per replanning cycle, refined further in Section 25/35).
- **Path smoothness** — curvature and jerk bounded within vehicle-comfort limits.
- **Minimum clearance / time-to-collision** maintained above a safety threshold during near-miss events (village road pedestrian, cattle crossing).
- **Scenario completion rate** — vehicle reaches its goal without disengagement/deadlock, including in dense, cluttered scenes (market).

8. Constraints

- 36-hour build window (SIH hackathon format) — see Section 38.
- MATLAB/Simulink + RoadRunner licensing must be available to the team (MathWorks normally provides SIH participants access; this must be confirmed early, not assumed).
- No real vehicle or real sensor hardware — everything is simulated (this is explicitly a simulation PS, which lowers risk but means "real-world" claims must be scoped as simulated results only).
- Team skill constraints: MATLAB/Simulink proficiency, not just Python/ROS, is required to use the encouraged toolchain effectively.

9. Explicit Facts vs. Assumptions

Explicitly stated by PS:

- Must use a multi-sensor perception setup (camera, LiDAR, radar).
- Must validate on ≥ 5 named scenario types, with ≥ 2 detailed RoadRunner scenes (village road, urban intersection).
- Must report replanning latency, path smoothness, scenario completion rate.
- Must produce simulation model, scenarios, report, and demo video.
- MathWorks tools are *encouraged*, not mandatory.

Strongly implied:

- The system should run as a genuine closed loop (sense $\rightarrow$ predict $\rightarrow$ plan $\rightarrow$ act $\rightarrow$ re-sense), not open-loop scripted animation.
- "Real time" implies the replanning cycle should be fast enough to react before a collision becomes unavoidable at the scenario's operating speed — an exact latency number is not given.
- Judges will expect a rule-based or hybrid decision layer (Stateflow is explicitly named) rather than a purely black-box end-to-end policy, since explainability is implicitly valued by naming Stateflow.

Assumptions requiring validation (never silently treated as fact in this document):

- That Apollo/Autoware would *fail* (rather than merely underperform) on Indian scenarios — unverified; no direct benchmark exists for this claim, and this document explicitly does not assert it.
- That MathWorks will provide free/adequate compute+licenses for the team during SIH — depends on the specific SIH-MathWorks arrangement and must be confirmed with organizers.
- That IDD/IDD-3D/DriveIndia/TraPHic data can be legally reused for a derivative simulation-training pipeline within the hackathon's time and licensing constraints — must be checked per dataset license before use.

10. Core Problem Sentence

(Restated for reference — see Section 2.) *Lane-centric AV planning fails to generalize to unstructured, heterogeneous Indian traffic; our system must demonstrate, in closed-loop simulation, that a non-lane-dependent perception–prediction–planning–control pipeline can navigate five representative Indian scenarios safely and replan in real time.*

11. Research Questions

1. How do occupancy-grid/costmap-based planners compare to lane-graph-based planners when lane geometry is unavailable or unreliable?
2. How much does heterogeneous-agent-aware prediction (TraPHic/TrafficPredict-style) improve safety margins over agent-agnostic constant-velocity prediction, for the specific agent mix named in the PS?
3. What is the minimum sensor/perception fidelity (2D+radar vs. full 3D LiDAR) that is *sufficient*, not maximal, for the five named scenarios?
4. What planning algorithm(s) remain real-time-feasible at the replanning frequencies needed for sudden events (cattle crossing, sudden pedestrian movement)?
5. What decision-layer structure (Stateflow FSM vs. behavior tree) is both explainable to judges and implementable within 36 hours?

12. Indian Road Context

Condition	Existing assumption in standard AV research	Why it becomes difficult	Existing research response	Remaining limitation
No/unclear lane markings	Lane detection + HD map localization	Drivable-area boundary is ambiguous or absent	Free-space/drivable-area segmentation (IDD's added "drivable area" class) IDD identifies new classes such as drivable areas besides existing classes)	Still relies on segmentation accuracy in cluttered scenes; no standard for how planners should consume free-space output
Heterogeneous agent mix (2/3/4-wheelers, pedestrians, animals)	Homogeneous vehicle-only or car/pedestrian/cyclist-only prediction	Different agents have very different kinematics and behavior norms	Category-aware interaction models (TrafficPredict, TraPHic) TrafficPredict integrates prediction for different agent categories using a category-aware attention layer)	Two-wheelers remain the hardest agent type to predict even with modern heterogeneous benchmarks two-wheelers emerge as the most challenging agent type for accurate prediction in heterogeneous traffic)
Informal merging /	Signaled lane	Agents merge without	Behavior/graph-based interaction modeling (Social-GAT, graph convolution approaches) a Social-GAT-based	Requires interaction labels/annotations that are costly to collect at scale heterogeneous trajectory prediction methods typically rely

lane discipline absence	changes, rule-following agents	signaling or lane discipline	framework was proposed specifically to capture the interaction and motion characteristics of powered two-wheelers in mixed traffic)	on extra class-label information that is costly to annotate and does not generalize to different behaviors within the same class)
Animal crossing	Not modeled at all in most AV research	Animals are rare-class, unpredictable, and safety-critical	Dedicated Indian roadway-animal detection models a real-time detection method was developed specifically for Indian stray and wild animals to help prevent road accidents)	Small, narrow datasets (~1,600 images); accuracy still moderate (~84%) and not integrated with planning
Unsignalized/dense intersections	Traffic-signal-conditioned planning	No signal state to condition on; must infer right-of-way from agent behavior	Standard unsignalized-intersection negotiation research (general, not India-specific)	Largely transferable, but assumption of "few, orderly agents" at the intersection breaks down in dense Indian junctions

13. Existing Autonomous Driving Systems

Both Apollo and Autoware are full-stack, ROS/middleware-based systems intended for real or simulated vehicles, not lightweight simulation sandboxes (Apollo is an open-source autonomous driving software platform with a modular full-stack architecture spanning perception, localization, and planning, typically coupled with real vehicles or external simulators rather than being a simulator itself). Neither ships with Indian-context scenario libraries or Indian-trained perception models out of the box. Bringing either into a 36-hour hackathon means standing up ROS, message bridges, and simulator connectors before any Indian-specific work begins — a heavy tax on time that MATLAB/Simulink (already scenario-and-sensor-aware via RoadRunner + Automated Driving Toolbox) avoids.

14. Existing Solutions

Existing Solution	What It Solves	Approach	Strength	Limitation	Relevance to PS	Remaining Gap
Apollo (Baidu)	Full-stack planning/control for structured urban roads	Sequential scenario selection → rule-conditioned planning	Mature, production-grade, smooth trajectories Apollo.Auto's Public Road planner offered smoother trajectories and enhanced passenger comfort)	Higher compute cost; scenario library assumes structured road types	Reference architecture only; too heavy to fork for SIH	No native handling of non-lane-based heterogeneous agents
Autoware	Full-stack planning/control, safety-margin-favoring	Parallel scenario computation + selector	Larger obstacle clearance, more conservative/safe Autoware.AI's Open Planner provided higher safety margins by maintaining larger clearance distances from obstacles)	Longer paths, higher energy use; ROS/Ubuntu version constraints historically strict Autoware requires a specific ROS/Ubuntu combination and does not support newer OpenCV versions)	Same as Apollo — reference only	Same lane/scenario-centric limitation
MathWorks Automated Driving Toolbox reference examples	Costmap-based planning + lane-following controllers	Hybrid A*/costmap planning on cuboid or RoadRunner scenes	Directly usable inside the encouraged toolchain; documented, supported	Reference examples are structured-road-biased by default (lane following, parking)	High — this is our base platform	Must be extended with Indian-style occupancy costmaps and heterogeneous prediction, not used as-is
TraPHic / TrafficPredict (research prototypes, not deployable systems)	Heterogeneous-agent trajectory prediction, India-relevant	LSTM/attention over category-labeled agents	Demonstrated accuracy gains on exactly this traffic mix TraPHic demonstrates up to 30% improvement in prediction accuracy on dense heterogeneous traffic including animals and two/three-wheelers)	Prediction-only; not integrated with planning/control; needs category-labeled training data modeling heterogeneous constraints requires knowledge of the shapes and sizes of different road agents, which could be tedious to collect)	High conceptual relevance; informs our prediction module design	No existing open closed-loop system integrates this prediction style with planning

This table is deliberately not "everyone else is bad, we are good" — Apollo and Autoware are strong, production-grade systems; the gap is specifically their fit to unstructured Indian roads within a MATLAB/Simulink-based hackathon deliverable, not general competence.

15. Research Paper Analysis

#	Paper	Authors / Year / Venue	Problem	Method	Dataset	Key Result	Limitation
1	TraPHic: Trajectory Prediction in Dense and Heterogeneous Traffic Using Weighted Interactions	(2018/2019, arXiv 1812.04767)	Predicting trajectories in dense, heterogeneous (Indian-style) traffic	LSTM-CNN hybrid with horizon-based and heterogeneity-based weighted interactions	Custom dense-traffic dataset + standard benchmarks	Up to 30% improvement in prediction accuracy over prior SOTA on heterogeneous traffic We observe up to 30% improvement in prediction accuracy)	Needs agent shape/size ground truth; weaker on sparse/homogeneous traffic we do not outperform prior methods on sparse or homogeneous traffic videos)
2	TrafficPredict: Trajectory Prediction for Heterogeneous Traffic-Agents	(2018/2019, arXiv 1811.02146)	Joint prediction across pedestrians/bicycles/vehicles	LSTM with a category-aware self-attention layer	Mixed real-world traffic video	Lower average/final displacement error vs. RNN-ED, Social-LSTM, Social-Attention baselines TrafficPredict achieves the lowest average and final displacement errors compared to prior baselines across pedestrian, bicycle, and vehicle categories)	Category-based, needs labeled age type; not tested on Indian-specific animal/cart classes
3	HetroD: A High-Fidelity Drone Dataset and Benchmark for Autonomous Driving in Heterogeneous Traffic	(2026, arXiv 2602.03447)	Benchmarking planning+prediction degradation under increasing traffic heterogeneity	Drone-derived HD-map dataset with VRU-rich trajectories	65.4k VRU-rich trajectories	Quantifies that state-of-the-art forecasting/planning degrades as heterogeneity rises; two-wheelers are the hardest class; rule-based planners show high lateral VRU collision rates rule-based planners demonstrate disproportionately high lateral VRU collision rates despite maintaining strong lane-keeping performance)	Not India-specific; drone-derived, no ego-vehicle sensor data
4	IDD-3D: Indian Driving Dataset for 3D Unstructured Road Scenes	Dokania, Abdul Hafez, Subramanian, Chandraker, Jawahar (2023, WACV)	Lack of 3D multi-modal data for unstructured Indian roads	Multi-camera + LiDAR capture, 3D bounding-box annotation	12–15.5k annotated frames, 10 primary + 7 misc. categories	Establishes benchmarks for 3D detection/tracking in unstructured layouts; explicitly includes animals and unconventional vehicles as labeled classes	Detection/tracking only, no planning ground truth; India specific geography (Hyderabad-area capture)
5	IDD: A Dataset for Exploring Problems of Autonomous Navigation in Unconstrained Environments	Varma et al. (2018)	Segmentation-oriented AV research lacked unstructured-road data	10,004 finely annotated images, 34 classes, 182 sequences	—	Established that Indian road-scene label distributions and within-class diversity differ significantly from Cityscapes-style datasets it also reflects label distributions of road scenes significantly different from	2D image-only, no LiDAR/3D, no planning

						existing datasets, with most classes displaying greater within-class diversity)	
6	The Dynamic Window Approach to Collision Avoidance	Fox, Burgard, Thrun (1997)	Real-time reactive collision avoidance for mobile robots	Search directly in velocity space under kinematic constraints	—	Foundational reactive local planner still cited as a baseline/component in current AV/UGV research; it searches for commands controlling translational and rotational velocity directly in velocity space, correctly incorporating the vehicle's dynamics	Reactive-only; car fall into local minim no explicit multi-agent prediction
7	Path planning algorithms in the autonomous driving system: A comprehensive review	(2024, Mechatronics/ScienceDirect)	Comparing planning algorithm families for AV use	Structured literature review of search, sampling, and hybrid methods	—	Documents hybrid RRT-DWA and neural-RRT* variants that combine global search with local reactive avoidance RRT-DWA is a hybrid approach where RRT finds the global path and DWA calculates velocity, producing smoother and faster results than Dijkstra-DWA and A*-DWA)	Survey-level, not a novel method
8	Open-Source Autonomous Driving Software Platforms: Comparison of Autoware and Apollo	Jung, Paek, Kong (2025, arXiv 2501.18942)	Head-to-head technical comparison of the two leading open stacks	Module-by-module architectural + middleware comparison	—	Shows both stacks are architected around scenario-selection over a lane/road-type library Autoware computes multiple scenarios in parallel and activates the appropriate one; Apollo integrates regulations before selecting a single scenario)	Focuses on architecture, not on Indian/unstructure road performance directly

16. Research Gap

Existing heterogeneous-traffic trajectory prediction (TraPHic, TrafficPredict) already demonstrates that category-aware, non-lane-based prediction models outperform lane-based baselines on exactly the Indian-style agent mix named in the PS. Existing full AV stacks (Apollo, Autoware) already demonstrate competent planning and control, but are architected around lane/scenario selection and have been shown to still suffer degraded VRU-safety performance in heterogeneous traffic even outside the Indian context rule-based planners demonstrate disproportionately high lateral VRU collision rates despite maintaining strong lane-keeping performance). **No open, reproducible, closed-loop MATLAB/Simulink pipeline currently joins Indian-context perception classes, heterogeneous-agent-aware short-horizon prediction, and a non-lane-dependent planner into one validated system across the specific scenario set (unmarked village road, unsignalized dense intersection, highway slow-merge, dense market, sudden cattle crossing).** Our solution addresses this integration gap — not a prediction-accuracy gap or a planning-algorithm gap — by combining already-validated components (occupancy-grid planning, category-aware short-horizon prediction, Stateflow-based decision arbitration) into one demonstrable, metric-reported simulation.

17. Problem Gap vs. Technology Gap

- **Problem gap (real-world need, insufficiently addressed):** There is no publicly available, reproducible, closed-loop demonstration that ties Indian-road perception classes to real-time planning and control performance metrics. This is a genuine, evidenced gap.
- **Technology gap (why current tools can't fully solve it today):** Lane-graph-centric planning architectures (Apollo/Autoware) and lane-based prediction baselines are not *broken*, but they are optimized for a different assumption set; adapting them requires re-authoring their scenario/behavior layers, which is a significant engineering — not research — burden, especially within 36 hours. The genuine technology gap is the **absence of a light integration layer**, not the absence of algorithms.

18. Path Planning Research

Approach	Static obstacles	Dynamic obstacles	Compute cost	Real-time fit	Unstructured-road fit	Explainability	Implementation difficulty	SIH suitability
A* / Dijkstra (grid)	Good	Poor (needs replanning from scratch)	Low	Good for global, poor for reactive	Moderate (grid-based, lane-free)	High	Low	Good as a global planner layer
RRT / RRT*	Good, handles high-dim/nonholonomic	Requires re-sampling on change	Moderate–high	Moderate; RRT* convergence can be slow the highest cost rate is Basic RRT, then A-star with a lower rate, followed by Hybrid RRT-A* as the lowest)	High — no lane assumption, ideal for off-lane/unmarked segments	Moderate	Moderate	Good for village road/off-lane scenario specifically
Hybrid A*	Good, kinematically feasible paths	Needs periodic re-search	Moderate	Good — MathWorks' own reference planner uses this for unstructured environments Hybrid A* was implemented to generate kinematically feasible paths that handle nearby obstacles and virtual surfaces appropriately in unstructured environments)	High — designed explicitly for unstructured/unknown environments Hybrid A* addresses the non-holonomic requirement, guaranteeing kinematic constraints during forward search in continuous coordinates)	Moderate	Moderate (built into Navigation Toolbox)	Primary recommendation
Dynamic Window Approach (DWA)	Weak (local only)	Good, reactive	Low	Excellent	Moderate — needs a global planner alongside it	High	Low	Good as the <i>local/reactive</i> layer, paired with Hybrid A*
Potential fields	Moderate	Moderate	Low	Good	Weak alone (local-minima issues in dense clutter)	Moderate	Low	Supplementary only
Frenet / lattice planning	Good on structured roads	Good with prediction	Moderate	Good	Weak — inherently assumes a reference lane/centerline	High	Moderate	Not recommended : primary (assumes lane geometry)

MPC / trajectory optimization	Good	Good if obstacle constraints are convexified	High	Moderate—good with efficient solvers	Good, especially near-term	Moderate	High	Good as the tracking/control layer, not the global planner
Learning-based (RL/imitation) planning	Data-dependent	Data-dependent	Varies	Uncertain in 36h	Potentially strong but data-hungry and hard to validate/explain	Low	High	Not recommended : primary for a judge-defensib 36h SIH build

Selected approach: Hybrid A* (global, kinematically-feasible, unstructured-road-appropriate, and already a MathWorks/Navigation Toolbox reference pattern the current hybrid A* approach dynamically generates a search graph based on motion primitives that define the vehicle's motion constraints, using a custom cost function suited to unstructured environments)) for global path generation over an occupancy-grid/costmap representation, with a DWA-style local reactive layer for sudden dynamic events (cattle crossing, sudden pedestrian movement), tracked by a lightweight MPC or pure-pursuit controller. This is selected because it directly matches PS requirements (real-time replanning, unstructured roads, kinematic feasibility) rather than because it is popular.

19. Collision Avoidance

Foundational reactive methods remain the practical backbone for a 36-hour build: the **Dynamic Window Approach**, which searches for velocity commands directly in velocity space while correctly incorporating vehicle dynamics (Fox, Burgard & Thrun, 1997), and **Velocity Obstacles (VO)**, originally formulated by Fiorini and Shiller (1998) for motion planning in dynamic environments. We recommend a **costmap + time-to-collision (TTC) safety envelope** approach: each predicted agent trajectory inflates a risk zone on the occupancy grid; the local planner is barred from paths that intersect a zone within a TTC threshold. This is simpler to implement and defend than full control-barrier-function formulations, while still being technically grounded — the VO/DWA lineage (including later extensions such as reciprocal n-body collision avoidance, ORCA) is decades-deep and well-cited in robotics and AV literature.

20. Trajectory Prediction

Decision: hybrid, category-aware, short-horizon prediction — not a heavy deep model.

- **Classical baseline (must-have):** constant-velocity / constant-acceleration models with a Kalman Filter per tracked agent — fast, explainable, and adequate for short (1–2s) horizons for most agents.
- **Category-aware adjustment (should-have):** apply agent-specific motion priors (pedestrians: near-omnidirectional, low speed, high heading variance; two-wheelers: high heading variance, filtering/lane-splitting behavior; animals: near-random, very low speed but abrupt direction changes; auto-rickshaws/carts: constrained but non-lane-following). This directly reflects the finding that a single undifferentiated model underperforms on this mix vehicle trajectories are constrained by steering and acceleration bounds whereas pedestrian motion is more unconstrained and stochastic, so a unified framework must accommodate these heterogeneous characteristics, e.g. by first classifying agent type and then delegating to category-specific prediction).
- **Learned component (nice-to-have, time permitting):** a lightweight LSTM/GRU per-category short-horizon predictor trained or fine-tuned on IDD-3D/TraPHic-style data if time allows, following the TraPHic category-weighted-interaction pattern TraPHic models horizon-based and heterogeneous-based weighted interactions between road agents) — explicitly scoped as a stretch goal, not the demo-critical path.
- **Why not full graph/transformer-based prediction:** these need larger labeled datasets and longer engineering time than 36 hours allows, and are harder to explain live to judges; two-wheeler prediction remains the hardest agent type even for state-of-the-art heterogeneous models two-wheelers emerge as the most challenging agent type for accurate prediction), so added model complexity has diminishing returns for a hackathon-scale demo.

21. Sensor & Sensor Fusion Research

Sensor	Information	Problem solved	Limitation	Include?
Camera	Class, color/texture, lane/road-edge cues	Object classification (auto-rickshaw vs. cart vs. animal), free-space/drivable-area segmentation	No direct depth/range; weather/lighting sensitive	Yes — primary perception sensor
LiDAR	Precise 3D geometry, range	Robust obstacle localization independent of lighting; matches IDD-3D's own sensor setup IDD-3D uses six cameras and one LiDAR sensor to enable sensor-fusion-based applications)	Cost/compute-heavy in reality; in simulation this is a non-issue	Yes — for 3D occupancy grid construction
Radar	Robust velocity/range even in poor visibility	Reliable relative-velocity estimate for fast agents (vehicles, buses) at range	Low angular resolution; less useful for small/slow agents (pedestrians, animals)	Yes — supplementary, mainly for oncoming/highway-merge scenario

GNSS/IMU	Ego-localization	Needed for scenario/ego-pose ground truth in RoadRunner	Not the PS's focus	Include minimally for closed-loop ego-state, not as a research focus
----------	------------------	---	--------------------	--

Fusion strategy: late/track-level fusion (fuse after each sensor's independent detections are formed into tracks) rather than early/raw-data fusion — track-level fusion is what Automated Driving Toolbox's sensor fusion examples are built around, is far faster to implement in 36 hours, and is defensible since raw early-fusion is not required by the PS's stated goals. Not every scenario needs every sensor at full fidelity — e.g., radar contributes little in the dense/slow market scenario and can be deprioritized there, while camera+LiDAR dominate for the village-road and market scenarios.

22. Perception

Given the PS is a *simulation* task, "perception" here means simulated sensor models detecting/classifying ground-truth actors placed in RoadRunner scenes (Automated Driving Toolbox provides this directly), not training a detector from scratch on real images. Where a learned detector is used (e.g., to classify a RoadRunner-rendered camera frame), lightweight pretrained YOLO-family models are appropriate and consistent with current practice for Indian-road object classes the top-performing YOLO-family variant on Indian traffic-relevant categories achieved 78.7% performance in the DriveIndia benchmark), and for animal-specific classes specifically YOLOv5 combined with adaptively spatial feature fusion improved animal (cattle) detection accuracy).

23. Environment Representation

Representation	Fit for unstructured + heterogeneous + replanning
Lane graph / HD map	Poor fit — the PS's entire premise is that lanes are unreliable/absent
Object list only	Insufficient alone — no spatial context for planning
Point cloud (raw)	Too low-level for planning directly; needed upstream of occupancy grid
Occupancy grid / costmap	Best fit — lane-free, directly consumable by Hybrid A*/DWA, and matches Automated Driving Toolbox's own costmap-based planning pattern Automated Driving Toolbox lets you plan driving paths with vehicle costmaps and motion-planning algorithms)
Bird's-eye-view (BEV) semantic map	Good complement for visualization and for feeding a learned prediction module, but heavier to build than needed for the core deliverable

Decision: occupancy grid/costmap as the primary environment representation, augmented with a simple drivable-area/free-space mask (mirroring IDD's own added "drivable area" class IDD identifies new classes such as drivable areas)) rather than any lane-graph structure.

24. Behaviour Planning

Approach	Reliability	Explainability	Complexity	SIH fit
Finite State Machine / Stateflow	High	High	Low–moderate	Recommended — explicitly named in the PS
Behaviour Trees	High	Moderate–high	Moderate	Viable alternative, but no native MATLAB integration as clean as Stateflow
Pure rule-based (if/else in code)	Moderate	High	Low	Acceptable fallback but less demonstrable/visual for judges
Utility-based decision-making	Moderate–high	Lower	Higher	Not recommended — harder to defend/tune in 36 hours

Decision: Stateflow-based FSM with states such as `Cruise → Yield → SlowDown → Stop → EmergencyBrake → Overtake/Merge → RejoinPath`, transitions triggered by TTC/clearance thresholds from the collision-avoidance layer and by agent-category cues from prediction. This is explainable, visually demonstrable to judges (Stateflow charts render as diagrams), and matches the PS's own suggested tool.

25. Vehicle Dynamics

Decision: kinematic bicycle model, not full dynamic/tire-force modeling. The PS's goal is to demonstrate that a *planned path is followable* and that replanning/collision-avoidance logic works — not to validate vehicle handling at the tire-force level. A kinematic bicycle model (as used even in recent published closed-loop AV planning studies with MATLAB/Simulink and RoadRunner the Roadrunner environment is used to define the driving scenario and the Vehicle Dynamics Blockset is employed to simulate vehicle dynamics, with the relevant optimization problems solved for a mean planning time around 31 ms per cycle)) is sufficient, simulation-fast, and appropriately scoped; Vehicle Dynamics Blockset can be substituted later only if extra time and marginal realism are wanted.

26. Candidate Technical Approaches (Summary)

1. **Recommended:** MATLAB/Simulink + RoadRunner, occupancy-grid representation, Hybrid A* (global) + DWA (local reactive), category-aware hybrid classical/learned prediction, Stateflow decision layer, kinematic bicycle model + pure-pursuit/MPC tracking.
2. **Alternative A (heavier):** Fork Autoware/Apollo and retrofit an occupancy-grid planner in place of their lane-based planner — technically stronger long-term but infeasible within 36 hours given ROS/middleware setup overhead.
3. **Alternative B (lighter but weaker):** Pure Python + CARLA with reactive-only (DWA/potential-fields) planning and no dedicated Indian-context prediction — faster to prototype but does not use the PS's encouraged toolchain and is weaker on the "adaptive"/prediction-driven replanning requirement.

We select **Option 1** and justify the full stack in Section 27–28.

27. Technology Stack Comparison

Requirement → Technology → Justification → Alternative → Trade-off → Decision

Requirement	Technology	Justification	Alternative	Trade-off	Decision
Author unstructured Indian road scenes (village road, unmarked edges, potholes)	RoadRunner	Native support for custom road damage/potholes and non-standard geometry via OpenCRG RoadRunner supports creating synthetic road geometries adding speedbumps, potholes, and other surface features)	CARLA town editor / hand-built OpenDRIVE	CARLA is free and Python-native but has weaker native support for informal/unmarked Indian-style geometry	RoadRunner
Simulate multi-sensor perception (camera/LiDAR/radar) and fuse tracks	Automated Driving Toolbox	Built-in sensor models + fusion + costmap planning examples Automated Driving Toolbox integrates scenarios, sensors, and vehicle dynamics for validating ADAS algorithms)	Custom Python sensor simulation	Custom sim gives more control but costs significant build time	Automated Driving Toolbox
Global + local path planning with replanning	Navigation Toolbox	Native search/sampling planners, path-following, and replanning tools Navigation Toolbox includes customizable search and sampling-based path planners and supports global and local path planning, path following, and obstacle avoidance)	Open-source OMPL / custom RRT* implementation	OMPL is more flexible for research-grade tuning but not integrated with RoadRunner/Simulink out of the box	Navigation Toolbox
Explainable behavior arbitration	Stateflow	Explicitly named by PS; visual, testable, integrates with Simulink	Behavior trees (custom)	Behavior trees are more scalable for very large behavior sets but overkill here and less native to MATLAB	Stateflow
Vehicle motion realism	Kinematic bicycle model (Simulink)	Sufficient fidelity for path-following demonstration at low added complexity	Vehicle Dynamics Blockset (full dynamics)	Full dynamics adds realism but adds tuning time disproportionate to the PS's actual ask	Kinematic bicycle model, Vehicle Dynamics Blockset as stretch goal
Detection/trajectory-prediction models	Deep Learning Toolbox (lightweight CNN/LSTM)	Named by PS; integrates with Simulink for closed-loop testing	Python (PyTorch) models called via MATLAB's Python interface	Python models may be faster to prototype if the team is more Python-fluent	Deep Learning Toolbox, with Python interop as fallback

28. Final Technology Stack

- **Scenario authoring:** RoadRunner (2 detailed scenes: unmarked village road, unsignalized dense intersection) + RoadRunner Scenario for the other three (highway merge, market, cattle crossing), reusing/adapting scene assets to manage time.
- **Sensing & fusion:** Automated Driving Toolbox sensor models (camera, LiDAR, radar) with track-level fusion.
- **Environment representation:** Occupancy grid / costmap + free-space mask.

- **Prediction:** Category-aware classical (Kalman-filter-per-class) baseline; optional lightweight learned LSTM stretch goal.
- **Global planning:** Hybrid A* (Navigation Toolbox).
- **Local/reactive avoidance:** DWA-style velocity-space search + TTC safety envelope.
- **Decision layer:** Stateflow FSM.
- **Control/vehicle model:** Kinematic bicycle model in Simulink, pure-pursuit or simple MPC tracking.
- **Language/tooling:** MATLAB/Simulink throughout, as encouraged and as best matches the toolbox ecosystem above; Python only as a fallback interop path for any stretch-goal learned model.

29. Final System Architecture

```

flowchart LR
    ENV[Environment / RoadRunner Scene] --> SENS[Sensors: Camera, LiDAR, Radar]
    SENS --> PERC[Perception: Detection + Classification]
    PERC --> FUSE[Track-Level Sensor Fusion]
    FUSE --> ENVMODEL[Environment Model: Occupancy Grid + Free-Space Mask]
    FUSE --> PRED[Category-Aware Short-Horizon Prediction]
    ENVMODEL --> BEHAVIOR[Stateflow Behaviour Decision]
    PRED --> BEHAVIOR
    BEHAVIOR --> PLAN[Hybrid A* Global Planner]
    PLAN --> LOCAL[DWA / TTC Local Collision Checking]
    LOCAL --> CONTROL[Pure-Pursuit / MPC Tracking]
    CONTROL --> DYN[Kinematic Bicycle Model]
    DYN --> ENV
    LOCAL -.replan trigger.-> PLAN

```

For each component: input, processing, output, algorithm, technology, and failure cases are detailed in Sections 18–25 above rather than repeated here.

29.1 Requirement Traceability Fixes — Architecture-Level Additions

The following three additions close explicit gaps against the PS text. They are framed here as **parity fixes, not differentiators** — each closes a specific line in the PS description that the architecture in Section 29 did not yet visibly cover.

A1 — Road-Surface Hazard Layer (Potholes). Priority: Core. Traced to the PS's own language: potholes are common and road edges are unclear. Previously, potholes existed only as RoadRunner scene geometry (visual, via OpenCRG surface authoring — Section 32) and were not a planning input. This addition extends perception/fusion to classify road-surface hazards (potholes, damaged patches, unclear edges) and feeds them into the costmap as high-cost or non-drivable cells, treated exactly like a static obstacle. *Flow:* Pothole detected → surface-hazard classifier → high-cost region → merged into unified costmap → Hybrid A*/DWA routes around it. *Acceptance criteria:* In the village-road and market scenarios, the planned path maintains a minimum clearance of ≥0.5 m from all labeled hazard cells across N runs, with zero hazard-cell traversals.

A2 — Wrong-Way / Against-Traffic Vehicle Detection. Priority: Core. Traced to the PS's own language: vehicles that drive against traffic. For each tracked vehicle, the system estimates heading and compares it against the expected traffic direction for that road segment, flagging a vehicle as wrong-way if heading deviates beyond 120° from the expected direction for a sustained window (to avoid false positives from brief lateral wobble). No ML model is required for this — a heading-vs-road-direction comparison on the existing fused track state is sufficient for the prototype; this is explicitly a lightweight geometric check, not a stretch-goal ML task. *Flow:* Detect vehicle → estimate heading (from fused track) → compare to expected lane/road direction → wrong-way flag → elevate collision-risk weight for that agent in the costmap/TTC calculation → slow / avoid / replan. *Acceptance criteria:* In a scenario with a synthetic wrong-way vehicle injected, the system flags it within X frames of sustained wrong-heading motion (X to be fixed during implementation) and demonstrably increases following/passing distance versus an unflagged vehicle on the same trajectory.

A3 — Explicit Informal-Merge Detection Subsystem. Priority: Core. Traced to the PS's own language: autos and vehicles that merge without signalling. The existing prediction + TTC + Stateflow stack can technically handle this, but until now it was implicit — not a named capability that could be pointed to directly in a demo or report. This addition promotes informal-merge handling to a named subsystem with its own explicit flow. *Flow:* Detect nearby vehicle → track position + velocity → detect lateral movement toward ego path → predict potential path intersection → compute TTC/collision risk → Stateflow → YIELD/SLOW DOWN → local planner replan. *PRD framing:* We specifically handle unsignalled informal merging by detecting lateral movement of nearby vehicles, predicting whether their trajectory will intersect our planned path, and triggering a yield/slow-down/replan sequence when predicted TTC falls below a safety threshold. *Acceptance criteria:* In the highway-merge scenario, log and report the TTC value at the moment YIELD is triggered, and confirm the replanned path avoids the merging vehicle with a defined minimum clearance across N runs.

30. Closed-Loop Operation

Observe → Understand → Predict → Decide → Plan → Act → Observe Again → Replan, implemented as a fixed-rate Simulink simulation loop:

- **Perception update:** every sensor sample interval (simulated).
- **Prediction update:** every perception update, short-horizon (1–2s) per tracked agent.
- **Planning update:** on a timer *and* event-triggered (new obstacle enters a risk zone, TTC drops below threshold) — this dual trigger is our design decision, not a PS requirement, chosen to balance responsiveness with compute cost.

- **Control update:** highest frequency, tracking the latest committed trajectory.
- **Replanning trigger:** TTC below threshold, planned path blocked, or Stateflow transitions to a non-cruise state.
- **Emergency response:** a dedicated Stateflow `EmergencyBrake` state overrides the tracked trajectory directly via the control layer, bypassing full replanning latency for the cattle-crossing/sudden-pedestrian cases.

PS requirement vs. our design decision, made explicit: the PS requires "real-time replanning" and "closed-loop validation" (requirement); the specific event-triggered + timed dual-replanning strategy and the emergency-override Stateflow state are our design decisions, not stated in the PS.

31. Five Required Scenarios

	Scenario 1: Unmarked Village Road	Scenario 2: Unsignalized Urban Intersection	Scenario 3: Highway Merge, Slow Vehicles	Scenario 4: Dense Market, Mixed Traffic	Scenario 5: Sudden Cattle Crossing
Road geometry	No lane markings, narrow, unclear edges	Multi-way junction, no signals	Multi-lane highway with a merge point	Narrow, congested lane with stalls encroaching	Two-lane rural/semi-urban road
Actors	Pedestrians, occasional two-wheeler, pushcart	Cars, auto-rickshaws, two-wheelers, pedestrians crossing informally	Cars, slow trucks/tractors merging	Pedestrians, carts, two/three-wheelers, parked vehicles	Ego vehicle + grazing/crossing cattle
Main challenge	Free-space estimation without lane cues	Right-of-way inference without signals, high agent density	Speed differential + short merge window	Extreme clutter, frequent replanning, low speed	Rare-class detection + abrupt stop
Expected perception focus	Drivable-area/free-space segmentation	Multi-class detection at range + close range	Relative velocity estimation (radar-heavy)	Dense multi-class detection at close range	Rare-class (animal) detection reliability
Expected prediction focus	Pedestrian/pushcart low-speed, high-heading-variance model	Multi-agent simultaneous crossing prediction	Slow-vehicle constant-acceleration prediction	Category-specific short-horizon prediction under occlusion	Near-random low-speed animal motion model
Expected behaviour decision	Cruise → SlowDown	Yield/negotiate → Cruise	Follow → Merge	Continuous SlowDown/Yield cycling	Cruise → EmergencyBrake
Planning response	Hybrid A* over free-space costmap	Frequent replanning at short horizon	Longer-horizon planning with speed profile adjustment	High-frequency local replanning, low-speed profile	Emergency local replan / stop
Success criteria	Reach goal without leaving drivable area or contacting agents	Reach goal without collision or indefinite deadlock	Complete merge within a safe TTC margin	Reach goal with no collisions inside dense clutter	Full stop before contact, resume after clearance
Key metric	Path smoothness, free-space adherence	Scenario completion rate, negotiation time	Time-to-collision at merge, completion time	Replanning latency under high obstacle density	Minimum clearance / TTC, stop distance

Each scenario validates a distinct capability: (1) lane-free free-space planning, (2) multi-agent right-of-way inference, (3) predictive merging under speed differential, (4) high-frequency replanning under clutter, (5) rare-event emergency response — together covering the PS's full stated requirement set without redundancy.

32. Simulation Environment (RoadRunner Design)

Scene 1 — Indian village road: narrow single carriageway, unpaved/mixed-surface shoulders, no painted lane lines, occasional potholes (via OpenCRG surface authoring RoadRunner supports creating synthetic road geometries with speedbumps, potholes, and other surface features)), sparse roadside structures, 2–3 pedestrian/pushcart actors with scripted or MATLAB/Simulink-authored behavior.

Scene 2 — Indian urban intersection: 4-way junction with no traffic signals authored (or signals present but non-functional, matching real Indian conditions), dense actor mix (cars, auto-rickshaws, two-wheelers, pedestrians), informal crossing behavior authored via RoadRunner Scenario actor logic RoadRunner provides tools for setting and configuring traffic signal timing, phases, and vehicle paths at intersections) (deliberately configured as unsignalized/ignored for this scenario), randomized spawn timing across runs for the experimental methodology in Section 34.

Both scenes' ego vehicle carries the simulated camera+LiDAR+radar rig, integrated with the Simulink planning/control model via RoadRunner Scenario's MATLAB/Simulink actor-behavior interface use MATLAB and Simulink to customize the behavior of actors, including vehicles and pedestrians). Scenarios 3–5 are built as RoadRunner Scenario variants (highway template, market-density variant of an urban scene, rural-road variant with an animal actor) to manage the 36-hour budget rather than as fully bespoke scenes.

33. Dataset Analysis

Dataset	Source	Context	Sensors	Objects	Task	Size	Access	Relevance
IDD	IIIT Hyderabad / insaan.iiit.ac.in	Indian unstructured roads	Camera	34 classes incl. drivable area	Semantic segmentation	10,004 images, 182 sequences 10,004 images finely annotated with 34 classes collected from 182 drive sequences)	Public via project site	Class taxonomy + free-space definition reference
IDD-3D	IIIT Hyderabad / iHub-Data	Indian unstructured roads	6 cameras + 1 LiDAR	10 primary + 7 misc. classes incl. animals	3D detection/tracking	12–15.5k annotated frames 12k annotated driving LiDAR frames across various traffic scenarios)	Public code/data on GitHub	Reference for perception-module class list and validation
DriveIndia	TiHAN-IIT Hyderabad	Indian traffic, varied weather/lighting	Camera	24 categories	2D object detection	66,986 images, 120+ hrs, 3,400+ km 66,986 high-resolution images across 24 traffic-relevant categories collected over 120+ hours and 3,400+ kilometers)	Public via TiHAN repository	Larger-scale detection reference if time allows fine-tuning
TraPHic dense-traffic dataset	Research authors (UMD/UNC-affiliated)	Dense heterogeneous (Indian-style) traffic video	Camera (video)	Cars, buses, pedestrians, two/three-wheelers, animals	Trajectory prediction	Custom, paper-specific	Via paper/authors	Direct pattern reference for our prediction-module category weighting
Built-in MathWorks scenario/sensor datasets	MathWorks	Generic + RoadRunner sample scenes	Synthetic (camera/LiDAR/radar models)	Standard AV object classes	Scenario/sensor simulation	N/A (generative)	Included with toolbox	Primary source of <i>synthetic</i> training/test data for our closed loop, since simulation — not real datasets — is the PS's actual required deliverable

What is actually required: given the PS is a *simulation* deliverable, the built-in MathWorks synthetic scenario/sensor data is the primary and sufficient

dataset; IDD/IDD-3D/DriveIndia/TraPHic are used only as **class-taxonomy and behavior-pattern references** (to make our synthetic scenes and prediction priors realistic), not as training data that must be ingested end-to-end — this keeps scope realistic for 36 hours and avoids license/logistics risk from external datasets.

34. Experimental Methodology

- **Baseline vs. proposed:** run each of the 5 scenarios with (a) a naive baseline (lane-following/constant-velocity-prediction planner, representing the "current solution" critiqued in Section 6) and (b) our proposed occupancy-grid + category-aware-prediction + Hybrid A* pipeline.
- **Multiple randomized runs:** ≥ 5 runs per scenario per configuration, varying actor spawn timing/position within realistic bounds.
- **Variation axes:** traffic density (sparse/dense), agent speed variation, and — time permitting — synthetic sensor noise injection on detections.
- **Reporting:** mean $\pm$ spread (not single-run point estimates) for every metric in Section 35, explicitly flagging where sample size is too small (a hackathon-scale honesty point that strengthens judge defense, per Section 45's guidance to acknowledge limitations).

35. Evaluation Metrics

Safety: collision rate, collision-free completion rate, minimum clearance, time-to-collision, near-miss count. **Planning:** replanning latency (ms/cycle), planning frequency, path length, scenario completion rate. **Smoothness:** curvature, longitudinal/lateral acceleration, jerk. **Driving quality:** travel time, progress-to-goal, emergency-braking count, comfort (jerk-derived proxy). **Robustness:** performance under traffic-density variation and (if implemented) sensor-noise variation.

Essential for the final demo (given 36-hour constraint): collision-free completion rate, replanning latency, minimum clearance/TTC, and path smoothness — these four map directly onto the PS's own named metrics (replanning latency, path smoothness, scenario completion rate) plus one added safety metric (TTC/clearance) that we consider indispensable for judge credibility even though the PS did not name it explicitly.

35.1 A4 — Mandatory Empirical Validation Protocol (Priority: Core)

Traced to the PS's own "Expected Solution" section: performance metrics, scenario completion rate, and a demonstration video are all named requirements. As of this PRD revision, architecture-level coverage of every requirement exists, but there are **no actual run results yet** — this is currently the largest practical gap in the submission and the single most load-bearing addition in this document.

For each of the five required scenarios (unmarked village road, unsignalled urban intersection, highway merge, dense market, cattle-crossing), the system must be run N times ($N = 10$ as a working default) with the following recorded per run:

- Collision-free completion (yes/no)
- Minimum clearance achieved
- Time-to-collision (TTC) at closest approach
- Replanning latency
- Path smoothness (jerk- or curvature-based metric)
- Scenario completion rate (% of runs completed without failure)
- Emergency-braking event count

PRD statement (explicit, to be carried into the report and judge Q&A): architectural coverage of a requirement is not treated as equivalent to validated performance. Only measured results across all five scenarios, per the protocol above, support a performance claim. Every other section of this document that describes a capability should be read as *design intent* until this protocol has produced results against it.

36. Baseline Comparison

- **Baseline 1 — Lane-dependent planner:** a Frenet/lane-centerline-based planner run on the same unstructured scenes (expected to degrade or become undefined where no lane centerline exists) — this directly operationalizes our stated failure condition rather than asserting it rhetorically.
- **Baseline 2 — Non-predictive reactive planner:** DWA-only with constant-velocity assumption and no category-aware prediction — isolates the specific contribution of our prediction module.
- **Baseline 3 — Static-obstacle-only planner:** ignores agent motion entirely — establishes a safety-margin floor to show why dynamic prediction matters at all.

Each baseline isolates exactly one of our design decisions (lane-freedom, category-aware prediction, dynamic-obstacle handling), so any measured improvement can be attributed to a specific, named component rather than to the system "in general."

37. Prototype / MVP

Already built (by the time of judging, expected): RoadRunner scenes 1 and 2, sensor+fusion pipeline, occupancy-grid construction, Hybrid A* + DWA planner, Stateflow FSM, kinematic bicycle model closed loop — this is the demonstrable core.

Must build: category-aware classical prediction module; scenario variants 3–5; metrics logging and the baseline-comparison harness.

Can build if time allows: lightweight learned (LSTM) prediction stretch goal; Vehicle Dynamics Blockset upgrade; sensor-noise robustness runs.

Future scope (explicitly not attempted in the hackathon): real-sensor/hardware-in-the-loop testing, large-scale learned end-to-end prediction trained on IDD-3D/DriveIndia, formal safety certification, multi-vehicle/V2X cooperative extensions.

38. 36-Hour Execution Plan

0–12 Hours: Set up RoadRunner + Automated Driving Toolbox + Navigation Toolbox environment; author Scene 1 (village road) and a base intersection scene; stand up sensor models and basic track-level fusion; get a first Hybrid A* path rendering on a static occupancy grid. *Risk:* license/toolbox environment setup delays — *backup:* pre-verify toolbox access before the event starts.

12–24 Hours: Add DWA local layer + TTC safety envelope; implement Stateflow FSM; implement classical category-aware prediction; close the loop with kinematic bicycle model + pure-pursuit control on Scene 1 and Scene 2. *Risk:* closed-loop timing/integration bugs — *backup:* fall back to a slower, non-real-time-strict loop for the demo if true real-time proves unstable, while being transparent about this to judges.

24–36 Hours: Build scenario variants 3–5, run the multi-trial experimental methodology, compute and log all metrics, run baseline comparisons, and assemble the report, PPT, and demo video. *Risk:* insufficient time for all 5 scenarios at full fidelity — *backup:* explicitly cut scenario fidelity (e.g., reuse Scene 2 geometry with a market-density actor overlay for Scenario 4) rather than cutting scenario count, since the PS requires all five.

39. Feasibility

- **36-hour demo feasibility:** Yes, for the "must build" scope in Section 37, using primarily MathWorks-native components rather than custom-built subsystems.
- **Team skill fit:** Requires MATLAB/Simulink/Stateflow competence; if the team is stronger in Python/ROS, Alternative B (Section 26) should be seriously reconsidered instead of forcing an unfamiliar toolchain.
- **Cost:** MATLAB/Simulink/RoadRunner licensing — typically provided free to SIH teams by MathWorks as PS owner; **must be confirmed with organizers, not assumed.**
- **API/tool failure fallback:** if RoadRunner access is unavailable, fall back to Automated Driving Toolbox's Cuboid/Driving Scenario Designer environment (lower visual fidelity, same underlying planning/control logic) — the PS explicitly allows non-RoadRunner scenario authoring as a fallback path.
- **ML underperformance fallback:** if the learned prediction stretch goal underperforms, the classical category-aware Kalman-filter baseline (Section 20) is already the "must-have" path, so this is a graceful, not catastrophic, degradation.

Must Have: closed loop across all 5 scenarios (even at reduced fidelity), core metrics, Hybrid A*+DWA+Stateflow, classical prediction. **Should Have:** full-fidelity RoadRunner scenes for all 5, baseline comparisons, robustness-under-density runs. **Nice to Have:** learned prediction stretch goal, Vehicle Dynamics Blockset upgrade, sensor-noise robustness. **Future Scope:** real hardware, large-scale learned end-to-end models, formal certification, V2X.

40. Cost

Simulation-only PS — no vehicle, sensor, or field-testing hardware cost. Primary cost is MathWorks tool licensing (expected to be provided for SIH participation) and compute (a standard laptop is sufficient for the scoped MVP; GPU is only relevant if the learned-prediction stretch goal is attempted).

41. Scalability

The occupancy-grid + Hybrid A* + category-aware-prediction architecture scales conceptually to more scenario types (additional road classes, additional agent categories) by extending the costmap inputs and prediction category set, without architectural rework — this is a genuine scalability property of the chosen design, not a marketing claim, since none of the core algorithms are scenario-hardcoded. Deployment-readiness claims beyond simulation (real sensors, certification, fleet-scale operation) are explicitly out of scope and should not be oversold to judges.

42. Novelty & Differentiation

- **Algorithmic novelty:** None claimed — Hybrid A*, DWA, Kalman-filter prediction, and Stateflow FSMs are all established techniques.
- **System integration novelty:** Joining an Indian-context-informed occupancy/free-space representation, category-aware prediction tuned to the PS's exact agent mix, and a Stateflow decision layer into one MATLAB/Simulink closed loop validated across the PS's specific five scenarios is the genuine (modest, honest) contribution.
- **Indian-context adaptation:** Using IDD/IDD-3D/TraPHic-informed class taxonomies and behavior priors (rather than importing an unmodified Western AV stack) is a real, defensible adaptation.
- **Evaluation novelty:** The explicit baseline-isolation methodology (Section 36), which attributes gains to specific design choices rather than to the system as a whole, is stronger practice than most hackathon submissions attempt.

43. Impact

Who benefits: AV developers and pilot programs targeting Indian deployment gain a reusable simulation benchmark pattern; road users benefit indirectly if such validation practices reduce unsafe deployment; MathWorks gains a demonstrated, Indian-context reference workflow for its toolchain. **How improvement is measured:** via the metrics in Section 35, compared against the named baselines — not via unsupported "X% safer" claims. **Deployment potential:** as a simulation/validation methodology, not as a production planner; real deployment would require the "future scope" items in Section 37 and is not something this prototype can claim to enable directly.

44. PPT Strategy (Five-Slide Framework)

1. **Problem Scale:** who (AV developers targeting India), where (village roads, unsignalized intersections, markets), why it matters (structured-road planners degrade on heterogeneous, non-lane-based traffic — cite TraPHic/HetroD evidence), with visible citations.
2. **Existing Solutions + Gap:** Apollo, Autoware, and published heterogeneous-prediction research — what they solve, where the *integration* gap specifically is (Section 16–17).
3. **Our Solution:** one sentence ("A MATLAB/Simulink closed-loop pipeline that replaces lane-graph planning with occupancy-grid Hybrid A* and category-aware prediction, validated on five Indian scenarios"), the architecture diagram from Section 29, and the explicit gap-to-solution link.
4. **Working Prototype:** live/recorded simulation across the five scenarios, metrics dashboard, explicit built-vs-planned breakdown from Section 37.
5. **Feasibility:** stack (Section 28), 36-hour plan (Section 38), cost (Section 40), and an honest deployment roadmap (Section 41/43) that does not overclaim beyond simulation.

45. Judge Questions & Answers

- **Q: Why not just use Apollo or Autoware, which already work?** A: Both are strong, but their planning modules are architected around scenario/lane selection Autoware and Apollo both select from a scenario library based on road/vehicle context); adapting either within 36 hours would consume most of our time on infrastructure rather than the Indian-context problem itself, and the PS's own encouraged toolchain (MathWorks) already gives us costmap-based planning natively.
- **Q: How do you know lane-based planners actually fail on Indian roads, rather than assuming it?** A: We do not claim proven outright failure — we cite quantified degradation in heterogeneous, VRU-dense traffic even outside India, where rule-based planners show disproportionately high lateral VRU collision rates despite strong lane-keeping performance (HetroD benchmark), and Indian-context datasets (IDD/IDD-3D) exist specifically because standard perception assumptions did not transfer, reflecting label distributions significantly different from existing Western datasets. We are explicit in this report about what is evidenced vs. assumed (Section 9).
- **Q: Why Hybrid A and not a learned end-to-end planner?** A: Explainability, real-time predictability, and 36-hour feasibility; learned planners need data and tuning time we do not have, and are harder to defend to judges component-by-component.
- **Q: Why does your prediction module only use classical/category-aware methods, not the newest transformer-based predictors?** A: Diminishing returns — even state-of-the-art heterogeneous predictors still struggle most on two-wheelers two-wheelers emerge as the most challenging agent type for accurate prediction), and the added engineering time is better spent completing the closed loop across all five required scenarios.
- **Q: Is your simulation representative of real Indian roads, or just a generic simulator with Indian labels?** A: We explicitly ground scene/actor design in Indian-context datasets and papers (IDD's class taxonomy, TraPHic's actor mix) rather than inventing scenarios from scratch — while being clear this is still a simulated, not field-validated, system.
- **Q: What happens if your system encounters an agent type it wasn't designed for?** A: The occupancy-grid/costmap layer is class-agnostic at the collision-avoidance level — even an unclassified object is still treated as an obstacle by the local DWA/TTC layer; only the *prediction* quality (not basic safety) degrades for unmodeled classes. This is a real robustness property of our layered design, and we say so plainly rather than overclaiming full robustness.

46. Risks / Red Flags

Problem	Why it is dangerous	Fix
Claiming Apollo/Autoware "fail" on Indian roads without direct evidence	Judges can trivially challenge an unsupported claim	State the evidenced, narrower claim: they are lane/scenario-centric and are <i>expected</i> to degrade, per HetroD-style heterogeneous-traffic evidence — not proven to fail on Indian roads specifically
Treating IDD/TraPHic-style datasets as if they were used for end-to-end training when they were only used as taxonomy/behavior references	Overclaiming data usage undermines credibility on inspection	Be explicit (as Section 33 is) that these are references, and that MathWorks synthetic scenario data is the actual training/testing substrate
Presenting a single successful run per scenario as "validated"	Fragile, statistically meaningless claim	Follow the multi-run methodology in Section 34 and report mean/spread
Treating Hybrid A*/DWA/Stateflow as	Judges will immediately spot established	Frame novelty correctly as integration + Indian-context adaptation

"our novel algorithm"	techniques mislabeled as innovation	(Section 42), not algorithmic invention
Promising real-world deployment readiness	This is a simulation PS; overclaiming deployment maturity is an easy red flag	Explicitly scope claims to simulation, with deployment listed only as future scope (Section 37/43)
Skipping baseline comparison	Without a baseline, "our system works" is unfalsifiable	Section 36's three isolated baselines directly address this
Treating architectural coverage (A1–A3, B1–B4) as proof the system is safer or more capable	Every addition below is currently a design/architecture claim, not a measured result — presenting it otherwise is the single easiest way to lose credibility under judge questioning	Enforce Section 35.1 (A4) as a hard gate: no capability claim is reported as "working" until it has run results across the five-scenario, N-run protocol; ablation/stress results (B5) should accompany any claim that an addition improves safety or robustness

47. What Still Needs Validation

- Whether MathWorks/SIH will actually provide adequate license/compute access for the team (organizational, not technical, risk).
- Whether the classical category-aware prediction module is "good enough" relative to the learned TraPHic-style approach for the PS's specific scenarios — only measurable once both are implemented and compared.
- Whether 36 hours is truly sufficient for all five full-fidelity RoadRunner scenes, or whether Section 38's scenario-reuse fallback will be required in practice.
- Real-world transferability of any conclusion — this entire analysis and prototype remain within simulation; no claim here should be read as validated against real Indian traffic.

48. Final Judge-Level Verdict

This PS is technically tractable within a 36-hour SIH format **if** scope is disciplined around the "must-have" list in Section 39 and the team commits to the MATLAB/Simulink-native stack rather than attempting to retrofit a heavier open-source AV stack. The genuine, evidence-backed contribution is integration and Indian-context adaptation of established techniques (Hybrid A*, DWA, category-aware Kalman-filter prediction, Stateflow arbitration) — not new algorithms — validated across the PS's five named scenarios with explicit baselines and honest metric reporting. The biggest execution risk is scenario-authoring time in RoadRunner, not algorithmic risk; the plan in Section 38 explicitly hedges against this. No claim in this document extends beyond what is evidenced in Sections 6, 9, and 15–16, and every technology choice in Section 27 is traceable back to a specific PS requirement.

49. Differentiation — Beyond Baseline (Part B)

Section 42 already frames our novelty honestly as integration and Indian-context adaptation rather than algorithmic invention. The five items below are what would move this submission from a checklist implementation to a genuinely differentiated one. They are kept in a distinct section deliberately: everything above this point is either PS-required coverage or an honest parity fix (Section 29.1, Section 35.1); everything below is optional depth that should only be claimed once built and validated per the A4/B5 protocol.

B1 — Dynamic Risk Map (replaces binary occupancy costmap). Priority: Core. Instead of a costmap that marks cells as simply occupied/free, build a continuous risk-valued map where every cell's cost reflects a blended risk score: static hazards (potholes, unclear edges — A1) plus dynamic agent risk (weighted by predicted position, uncertainty, and class — B2/B3) plus elevated risk for flagged wrong-way vehicles (A2). This makes risk, rather than a binary obstacle/no-obstacle judgment, the actual planning input — closer to how the PS itself frames the problem as one of uncertainty and mixed traffic, not clean obstacle avoidance. *Acceptance criteria:* Demonstrate that path choices visibly change (e.g., a wider berth) around higher-risk cells versus lower-risk cells that carry the same nominal "occupied" status.

B2 — Uncertainty-Aware Prediction. Priority: Core-to-stretch depending on time. Predicted agent trajectories should carry an uncertainty/confidence spread — even a simple growing-cone or 2–3 discrete-mode output — instead of a single deterministic trajectory plus a fixed safety buffer. Indian road-agent behavior is inherently multimodal (a pedestrian may or may not cross), and a single best-guess trajectory with padding silently reintroduces the structured-road assumption the PS explicitly asks us to move away from. A simplified 2-mode version is achievable within scope; a full probabilistic model is a stretch goal. *Acceptance criteria:* Show that the risk-map width/shape around a given agent (B1) changes with that agent's predicted uncertainty — e.g., a pedestrian near a market edge versus one moving steadily down a clear path.

B3 — Behaviour/Intent-Aware Prediction (class-conditioned). Priority: Core. Prediction should be conditioned on agent class, not just kinematics — a cow, a pushcart, an auto-rickshaw, and a pedestrian should carry different motion priors, reaction-time assumptions, and gap-acceptance thresholds rather than one interchangeable "obstacle" model. This is largely already covered by the category-aware prediction module in the core architecture (Section 20); this item's practical effort is mostly documentation and demo emphasis — making explicit that the module is intent/behaviour-aware, not merely class-labelled — and serves as the deliberately lower-effort substitute for a full game-theoretic/interaction-aware model. *Acceptance criteria:* Demonstrate visibly different system behaviour toward a stationary cow versus a stationary auto-rickshaw at the same distance (e.g., different passing margin or speed reduction).

B4 — Context-Adaptive Driving Policy. Priority: Core. The driving policy (safety margins, speed limits, aggressiveness of yielding) should change based on road/traffic context — a village road, a dense market, and a highway merge should not use identical planning parameters. This is implemented as a Stateflow-level parameter set keyed to scenario/context type, feeding into the existing Hybrid A*/DWA/TTC stack — no new subsystem is required, only

parameterization. It is cheap to implement and directly demonstrable in the video (visibly different behaviour across the five scenarios), which strengthens the "adaptive" claim in the PS title itself. *Acceptance criteria:* Report the parameter differences (e.g., speed cap, following distance) used per scenario, and show the resulting behaviour differs accordingly in the demo video.

B5 — Ablation + Stress Testing. Priority: Core if time allows a lightweight version. Beyond running the five canned scenarios once, robustness should be tested via (a) an ablation showing performance with versus without the Part A/B additions, and (b) a stress test — e.g., increased agent density, simulated sensor occlusion, or a held-out/unseen scenario variant not used during tuning. This is the evidence layer for every claim made in Part B: it is what proves the approach actually improves safety/robustness rather than merely looking sophisticated on paper, and extends directly from A4's validation protocol using the same metrics under perturbed conditions and with components selectively disabled. Even one ablation plus one stress condition is far better than none, and is a strong differentiator in minimal form since most other teams will skip this entirely. *Acceptance criteria:* Report metric deltas (completion rate, near-miss rate, clearance) with and without each major addition, and report performance under at least one stress condition not present in the original five scenarios.

49.1 Integration Note — Unified Risk Representation

A1 (pothole hazard), A2 (wrong-way risk elevation), B1 (dynamic risk map), and B2 (uncertainty-aware prediction) are not four separate patches — they are four inputs into a single, unified probabilistic risk representation that Hybrid A/DWA consumes. Static hazard cost (A1), dynamic agent risk shaped by class and uncertainty (B2/B3), and elevated cost for flagged wrong-way agents (A2) should all be written into the same risk-valued map (B1), not maintained as three or four separate costmap patches bolted onto the original binary occupancy grid. This should be presented in the report, PPT, and Q&A as **one coherent architectural upgrade** — the risk map is the single new object in the pipeline, and A1/A2/B2/B3 are simply its input channels. This framing is both more accurate to how the components should be built and easier to defend under judge questioning than a list of four unrelated bolt-ons.*

References

1. Dokania, S., Abdul Hafez, A.H., Subramanian, A., Chandraker, M., Jawahar, C.V. (2023). *IDD-3D: Indian Driving Dataset for 3D Unstructured Road Scenes*. WACV 2023 / arXiv:2210.12878. <https://arxiv.org/abs/2210.12878>
2. Varma, G. et al. (2018). *IDD: A Dataset for Exploring Problems of Autonomous Navigation in Unconstrained Environments*. <https://idd.insaan.iiit.ac.in/media/publications/idd-650.pdf>
3. DriveIndia dataset (TIHAN, IIT Hyderabad). <https://tihan.iith.ac.in/tiand-datasets/>
4. Chandra, R. et al. *TraPHic: Trajectory Prediction in Dense and Heterogeneous Traffic Using Weighted Interactions*. arXiv:1812.04767. <https://arxiv.org/pdf/1812.04767>
5. Ma, Y. et al. *TrafficPredict: Trajectory Prediction for Heterogeneous Traffic-Agents*. arXiv:1811.02146. <https://arxiv.org/html/1811.02146>
6. *HetroD: A High-Fidelity Drone Dataset and Benchmark for Autonomous Driving in Heterogeneous Traffic*. arXiv:2602.03447. <https://arxiv.org/pdf/2602.03447>
7. Golchoubian, M., Ghafurian, M., Dautenhahn, K., Azad, N.L. *Pedestrian Trajectory Prediction in Pedestrian-Vehicle Mixed Environments: A Systematic Review*. arXiv:2308.06419. <https://arxiv.org/pdf/2308.06419>
8. *Trajectory Prediction for Powered Two-Wheelers in Mixed Traffic Scenes: An Enhanced Social-GAT Approach*. *Systems*, 13(11):1036. <https://doi.org/10.3390/systems13111036>
9. Jung, H-Y., Paek, D-H., Kong, S-H. (2025). *Open-Source Autonomous Driving Software Platforms: Comparison of Autoware and Apollo*. arXiv:2501.18942. <https://arxiv.org/pdf/2501.18942>
10. *A Performance Evaluation of Open Source Autonomous Driving Frameworks: Case Studies of Apollo and Autoware*. IEEE. <https://ieeexplore.ieee.org/document/10918696/>
11. Fox, D., Burgard, W., Thrun, S. (1997). *The Dynamic Window Approach to Collision Avoidance*. *IEEE Robotics & Automation Magazine*, 4(1), 23–33.
12. Fiorini, P., Shiller, Z. (1998). *Motion Planning in Dynamic Environments Using Velocity Obstacles*. *International Journal of Robotics Research*, 17(7), 760–772.
13. Dolgov, D., Thrun, S., Montemerlo, M., Diebel, J. (2010). *Path Planning for Autonomous Vehicles in Unknown Semi-Structured Environments*. *International Journal of Robotics Research*, 29(5), 485–501.
14. *Path planning algorithms in the autonomous driving system: A comprehensive review*. *Mechatronics / ScienceDirect*. <https://www.sciencedirect.com/science/article/pii/S0921889024000137>
15. Sengan, S. et al. (2021). *Real-Time Automatic Investigation of Indian Roadway Animals by 3D Reconstruction Detection Using Deep Learning for R-3D-YOLOv3*. *Electronics*, 10(24), 3079. <https://doi.org/10.3390/electronics10243079>
16. MathWorks. *RoadRunner Product Page*. <https://www.mathworks.com/products/roadrunner.html>
17. MathWorks. *RoadRunner Scenario Documentation*. <https://www.mathworks.com/products/roadrunner-scenario.html>
18. MathWorks. *Automated Driving Toolbox*. <https://www.mathworks.com/products/automated-driving.html>

19. MathWorks. *Navigation Toolbox Documentation*. <https://www.mathworks.com/help/nav/index.html>

20. *Unifying Decision Making and Trajectory Planning in Automated Driving through Time-Varying Potential Fields* (RoadRunner + Vehicle Dynamics Blockset MATLAB/Simulink case study). arXiv:2603.13136. <https://arxiv.org/pdf/2603.13136>

Note: all findings above are attributed to the sources cited; where evidence was insufficient to support a claim (e.g., direct proof that Apollo/Autoware fail on Indian roads), this document says so explicitly rather than asserting it. This is a research and planning document for an SIH submission — it does not constitute a validated, deployment-ready engineering claim.